Universidad Tecnológica de Matamoros

Asignatura: Automatización de Infraestructura Digital I

Actividad 2.7: Gestión colaborativa de versiones con Git y GitHub

Profesor: Victor Alejandro Galvan Guerra

Integrantes:

Karina Hernandez Lona

Monserratt Guerra Cantu

Miranda Rodríguez Castellanos

Fabiola Rodríguez Rivera

Nahomi Itzel Rivas Martínez

Cuatrimestre: Septiembre – Diciembre 2026

Propósito: Que los estudiantes experimenten el funcionamiento de un sistema de control de versiones distribuido mediante Git y GitHub, identificando sus principales componentes y aplicando operaciones básicas para administrar cambios, consultar el historial y colaborar de manera organizada en un proyecto de software.

Organización:

Herramientas: Git, GitHub y Visual Studio Code.

Repositorio: cada equipo utilizará el repositorio existente automatizacion-redes.

Importante: Esta actividad no representa una nueva práctica del proyecto integrador. No deberán desarrollar funcionalidades adicionales para la automatización de redes ni modificar los productos correspondientes a las prácticas anteriores. Se trabajará exclusivamente con archivos de prueba para experimentar las operaciones de control de versiones.

Desarrollo de la actividad

Parte 1. Reconocimiento del repositorio

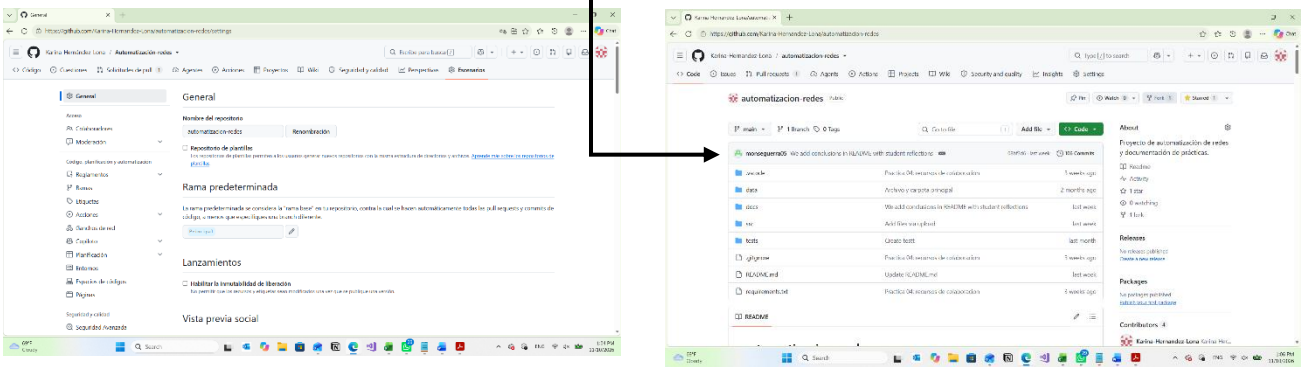
1. Repositorio automatizacion-redes en GitHub.

Nombre del repositorio: automatizacion-redes

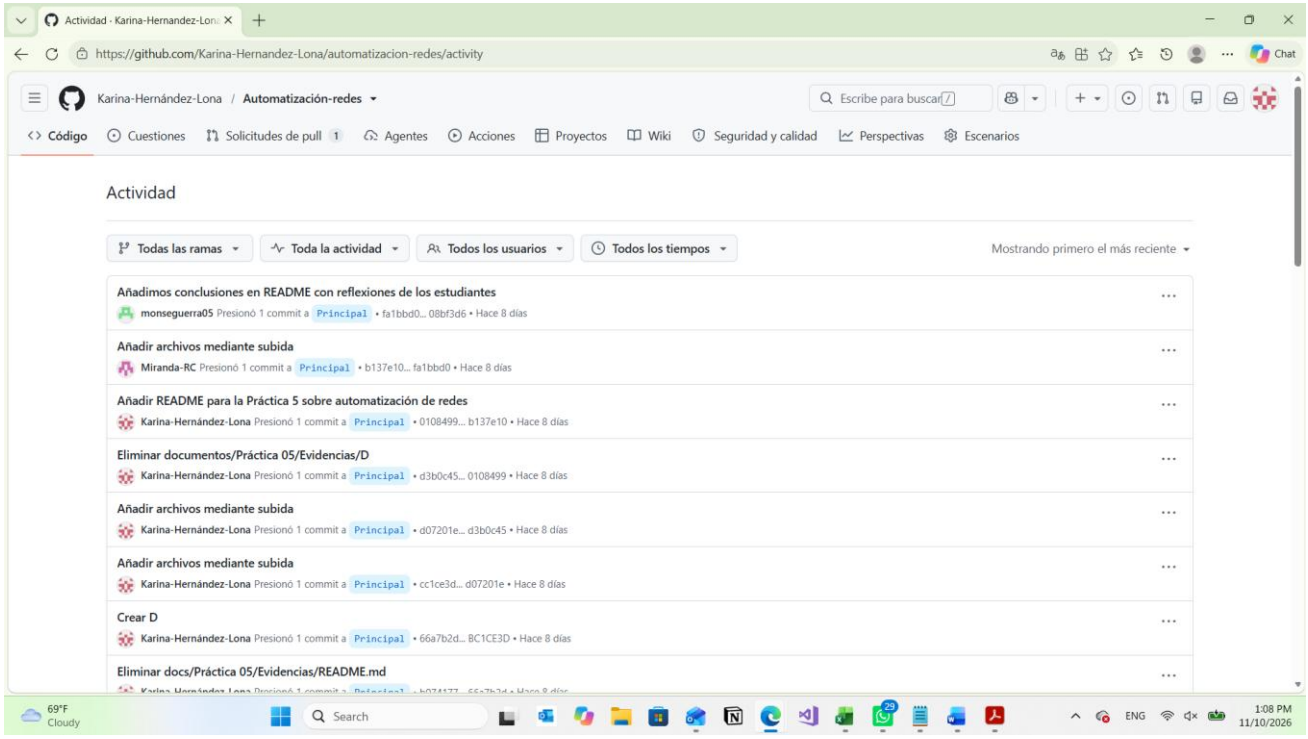
Propietario del repositorio: Karina-Hernández-Lona

Archivos y carpetas existentes.

Rama principal (main o master).



Historial de confirmaciones (commits).



Respondan en su cuaderno (se respondieron aquí y en el cuaderno, serán adjuntadas aparte en otro pdf):

a) ¿Qué información permite conocer el historial de cambios?

El historial de cambios es decir todos los (commits) nos permiten conocer las modificaciones realizadas en el proyecto, quién de las integrantes las hizo, cuándo se realizaron y qué archivos fueron modificados. También ayuda a identificar la evolución del proyecto y a recuperar versiones anteriores si es necesario así mismo dejar una descripción del porque se realizaron dichas modificaciones.

b) ¿Qué diferencia existe entre un repositorio local y uno remoto?

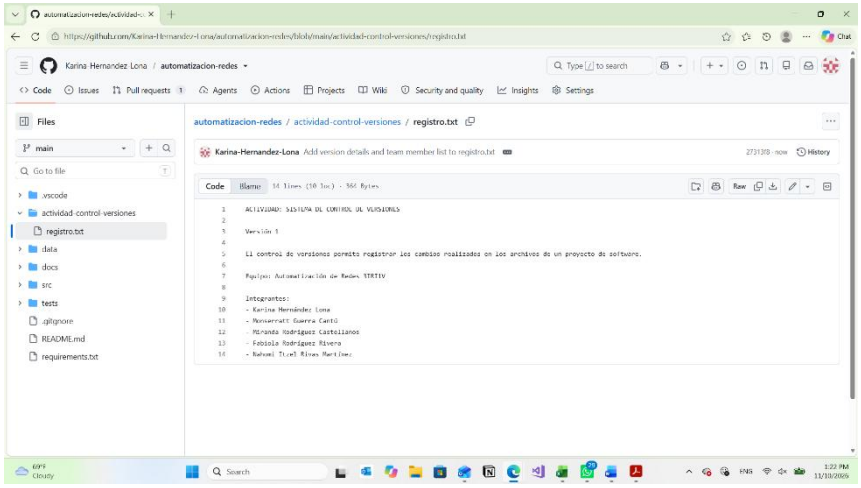
Lo que los diferencia es que un Repositorio local está almacenado en la computadora del desarrollador y permite trabajar en el proyecto sin conexión a Internet, mientras que un Repositorio remoto se encuentra alojado en un servidor o plataforma como GitHub y permite compartir el proyecto, respaldarlo y colaborar con otras personas.

c) ¿Por qué es importante conservar versiones anteriores de un proyecto?

Conservar versiones anteriores es sumamente importante porque permite recuperar trabajo en caso de errores, comparar cambios realizados a lo largo del tiempo, identificar cuándo

surgió un problema y mantener un registro de la evolución del proyecto. Además, facilita el trabajo colaborativo y el control de cambios.

Parte 2. Experimentación con el control de versiones local



Abran la terminal de Visual Studio Code y se ejecutaron los siguientes comandos:

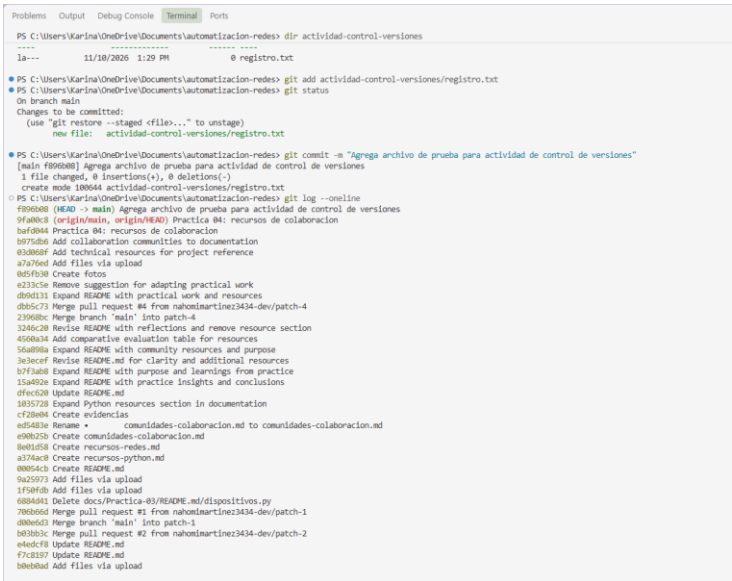
git status

git add actividad-control-versiones/registro.txt

git status

git commit -m "Agrega archivo de prueba para actividad de control de versiones"

git log --oneline



Reflexión: *¿Qué diferencia observaron entre ejecutar git add y git commit?*

La diferencia que observamos nosotras fue que **git add** solo prepara los archivos modificados para ser registrados, colocándolos en el área de preparación que investigando supimos que era (*staging area*). En cambio, **git commit** guarda esos cambios de forma permanente en el historial del repositorio mediante un mensaje descriptivo.

En esta práctica, al ejecutar git add el archivo registro.txt quedó listo para ser registrado, y al ejecutar git commit -m "Agrega archivo de prueba para actividad de control de versiones" se creó una nueva versión del proyecto que apareció en el historial al consultar con git log --oneline.

Parte 3. Identificación y recuperación de versiones

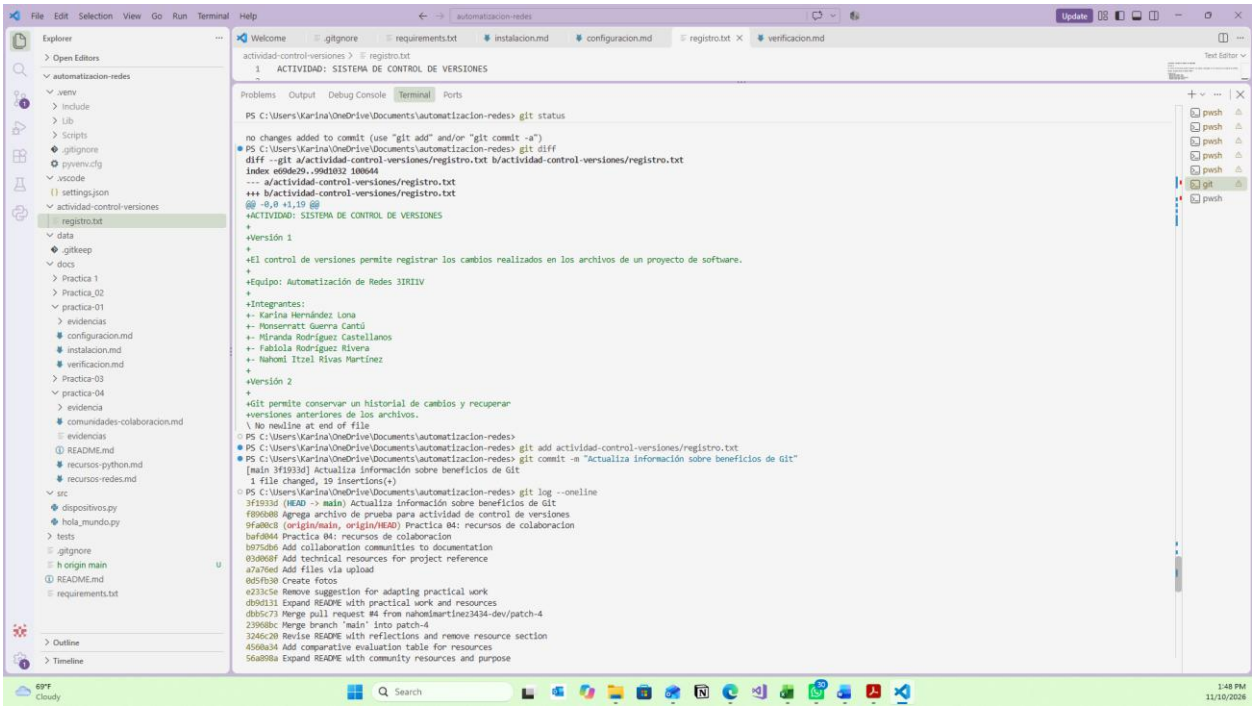
Comandos utilizados:

git diff

git add actividad-control-versiones/registro.txt

git commit -m "Actualiza información sobre beneficios de Git"

git log --oneline



Comparen los dos commits e identifiquen qué información conserva Git de cada versión.

Git conserva un registro completo de cada versión del archivo. En el primer commit se guardó la información inicial sobre el control de versiones y los integrantes del equipo. En el segundo commit se agregó la sección "Versión 2", donde se explica que Git permite conservar un historial de cambios y recuperar versiones anteriores de los archivos.

Además, Git almacena:

- El contenido de cada versión.
- La fecha y hora del cambio.
- El autor del cambio.
- El mensaje del commit.
- Un identificador único (hash) para cada commit.

Reflexión: *¿De qué manera esta característica puede ayudar cuando un cambio introduce un error en un proyecto?*

Esta característica permite revisar y recuperar versiones anteriores que funcionaban correctamente. Si un cambio introduce un error, es posible identificar cuándo ocurrió, comparar las diferencias entre versiones y restaurar una versión estable sin perder el historial del proyecto. Esto facilita la corrección de errores, evita la pérdida de información y mejora el control de los cambios realizados durante el desarrollo.

Parte 4. Experimentación con ramas Las ramas permiten trabajar en cambios

Comandos utilizados

git switch -c actividad/nueva-version 2

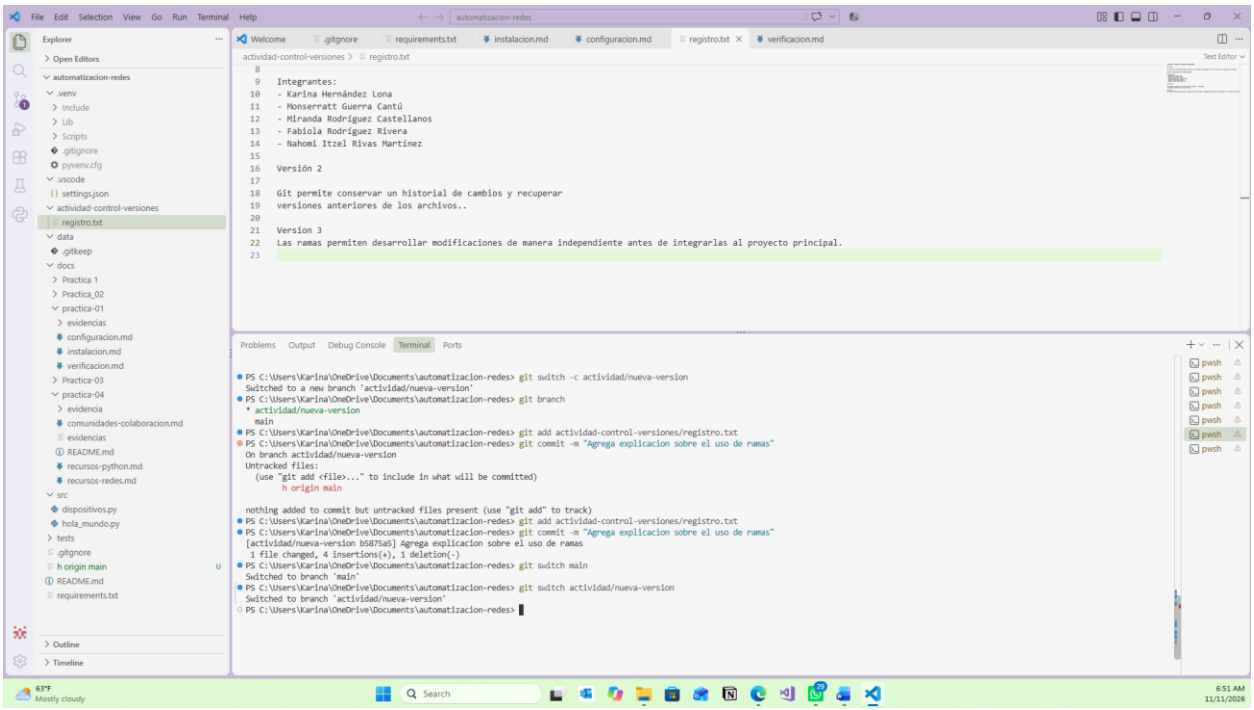
git branch

git add actividad-control-versiones/registro.txt

git commit -m "Agrega explicación sobre el uso de ramas"

git switch main

git switch actividad/nueva-version



Reflexión: *¿Por qué resulta conveniente trabajar con ramas cuando varias personas colaboran en un mismo proyecto?*

Resulta conveniente trabajar con las ramas ya que permiten que varias personas trabajen al mismo tiempo en diferentes partes del proyecto sin afectar el código principal, facilitando la colaboración y evitando conflictos.

Parte 5. Integración de cambios

git switch main 2

git merge actividad/nueva-version

git log --oneline --graph --all

git status

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\Karina\OneDrive\Documents\automatizacion-redes> git commit -m "Agrega explicacion sobre el uso de ramas"
 1 file changed, 4 insertions(+), 1 deletion(-)
PS C:\Users\Karina\OneDrive\Documents\automatizacion-redes> git switch main
Switched to branch 'main'
PS C:\Users\Karina\OneDrive\Documents\automatizacion-redes> git switch actividad/nueva-version
Switched to branch 'actividad/nueva-version'
PS C:\Users\Karina\OneDrive\Documents\automatizacion-redes> git switch main
Switched to branch 'main'
PS C:\Users\Karina\OneDrive\Documents\automatizacion-redes> git merge actividad/nueva-version
Updating 3f1933d..b5875a5
Fast-forward
 actividad-control-versiones/registro.txt | 5 ++++-
 1 file changed, 4 insertions(+), 1 deletion(-)
PS C:\Users\Karina\OneDrive\Documents\automatizacion-redes> git log --oneline --graph --all
* b5875a5 (HEAD -> main, actividad/nueva-version) Agrega explicacion sobre el uso de ramas
* 3f1933d Actualiza información sobre beneficios de Git
* f896b08 Agrega archivo de prueba para actividad de control de versiones
*   9fa00c8 (origin/main, origin/HEAD) Practica 04: recursos de colaboracion
|\
| * b975db6 Add collaboration communities to documentation
| * 03d068f Add technical resources for project reference
| * a7a76ed Add files via upload
| * 0d5fb30 Create fotos
| * e233c5e Remove suggestion for adapting practical work
| * db9d131 Expand README with practical work and resources
| *   dbb5c73 Merge pull request #4 from nahomimartinez3434-dev/patch-4
| |\
```

Conclusión en equipo:

Al finalizar esta actividad en donde trabajamos con Git, pudimos ver más claro cómo es que Git ofrece numerosos beneficios para la administración de proyectos de software, ya que facilita el control de versiones, el seguimiento de cambios y el trabajo colaborativo entre varios desarrolladores. Además, permite recuperar versiones anteriores del proyecto y mantener un historial organizado. El directorio de trabajo es donde se realizan las modificaciones a los archivos, el área de preparación (staging area) permite seleccionar los cambios que se incluirán en el siguiente commit, y el repositorio local almacena el historial completo de versiones del proyecto.

La diferencia entre un commit y una rama es que un commit guarda un conjunto específico de cambios en un momento determinado, mientras que una rama es una línea de desarrollo independiente que permite trabajar en nuevas características o correcciones sin afectar la versión principal.

Revisar los cambios antes de integrarlos es también sumamente importante para detectar errores, evitar conflictos y así mismo asegurar que el código mantenga su calidad y funcionamiento correcto por cada uno de los integrantes.

En futuros proyectos o actividades colaborativas, podemos aplicar lo aprendido utilizando ramas para trabajar en equipo de forma organizada, registrando cambios mediante commits y utilizando Git para coordinar, revisar e integrar el trabajo de todas nosotras de manera eficiente y sin necesidad de preocuparnos por perder alguna versión que fuera mejor.